

#). Transformer

Transformer is such type of NN which handle Sequence & Sequence task like

- Machine translation
- Question answering
- text summarization.

#). Why we need transformer :-

Attention mechanism solve the problem of improving the quality of translation but it fails in sequential training and why sequential training happens inherently in encoder & decoder have LSTMs which is accepting data in sequential manner which means we can't parallelize then training is slow if training is slow then we can't train model on huge datasets so transfer learning is not taking.

#). Advantages of transformer :-

- Scalability
- Transfer learning
- Multimodal Input & Output
- Flexible architecture
 - Encoder only transformer like - BERT.
 - Decoder only transformer like - GPT
- Ecosystem
 - hugging face.
 - blogs
- Easily to integrated with AI techniques
 - like → GANs + Transfer learning = Image Generator
 - Reinforcement learning + Transformers = Agents.

etc.

#> Disadvantages of Transformer

→ High computation (GPU)

→ Data LLMs

→ Overfitting

→ Energy consumption

→ Interpretability

→ Bias

#> Self Attention

NLP → words → number → vectorization

① One hot encoding:-

1. {mat cat mat} → sentence 1
2. {cat rat rat} → sentence 2

	mat	cat	rat
mat	1	0	0
cat	0	1	0
rat	0	0	1

num → [1,0,0] [0,1,0] [0,0,1]

② BOW Bag of words:-

	mat	rat	cat
<u>Sentence 1</u>	2	0	2
Sent 2	0	2	1

→ How many times words comes.

③ Word Embedding:-

↳ Capture semantic meaning

king → [0.9, 0.1, 1.0, 0.9]

queen → [0.9, 0.2, 0.4, 1.0]

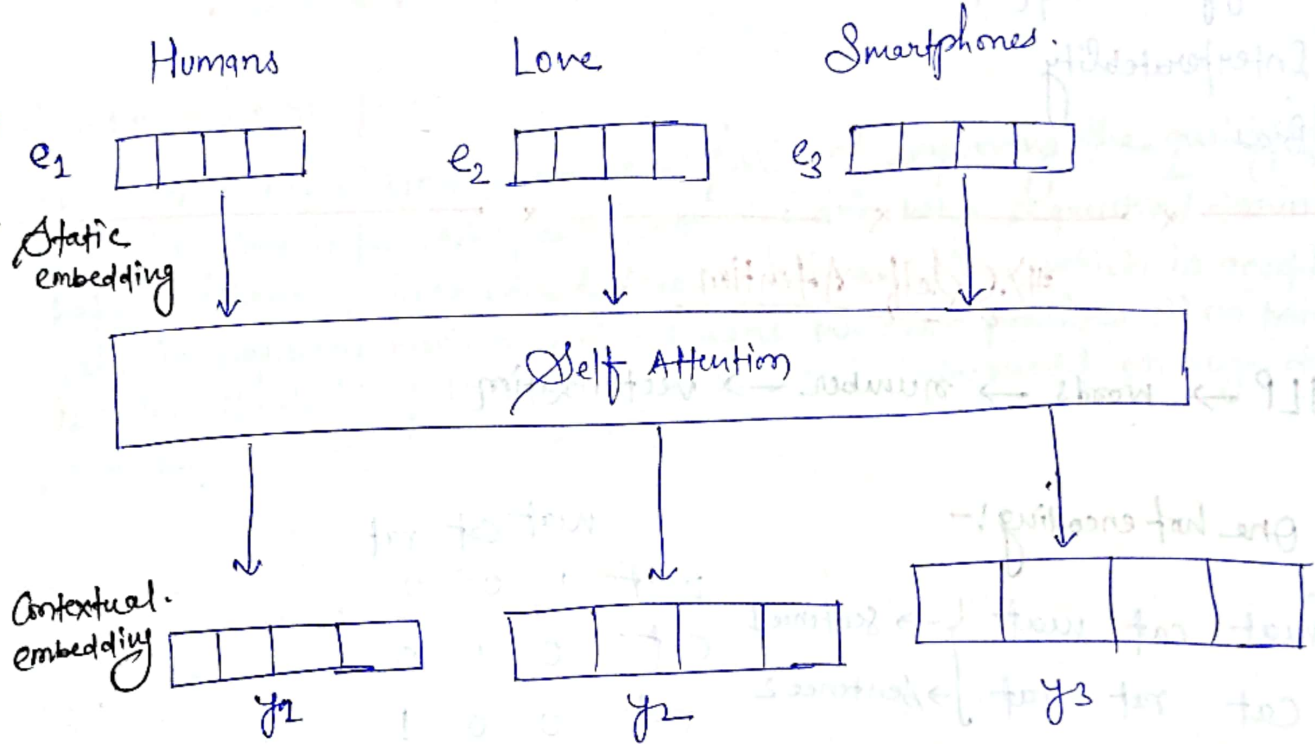
Similar

⇒ 2 similar words are looking similar vectors.

Word embedding is not only capture meaning, it capture Avg meaning.

→ Self Attention is a way it convert word embedding to dynamic contextual

Ex 1



#>. How Self Attention work :-

First Principle Approach:-

money bank grows

Sentence 1

money :- $0.7 \text{ money} + 0.2 \text{ bank} + 0.1 \text{ grows}$

bank :- $0.25 \text{ money} + 0.7 \text{ bank} + 0.05 \text{ grows}$

grows :- $0.1 \text{ money} + 0.2 \text{ bank} + 0.7 \text{ grows}$

river money bank flows.

Sentence 2

river :- $0.8 \text{ rivers} + 0.15 \text{ bank} + 0.05 \text{ flows}$

bank :- $0.2 \text{ rivers} + 0.75 \text{ bank} + 0.02 \text{ flows}$

flows :- $0.4 \text{ rivers} + 0.01 \text{ banks} + 0.59 \text{ flows}$

Similarity

$$e_{\text{money}} = e_{\text{money}}$$

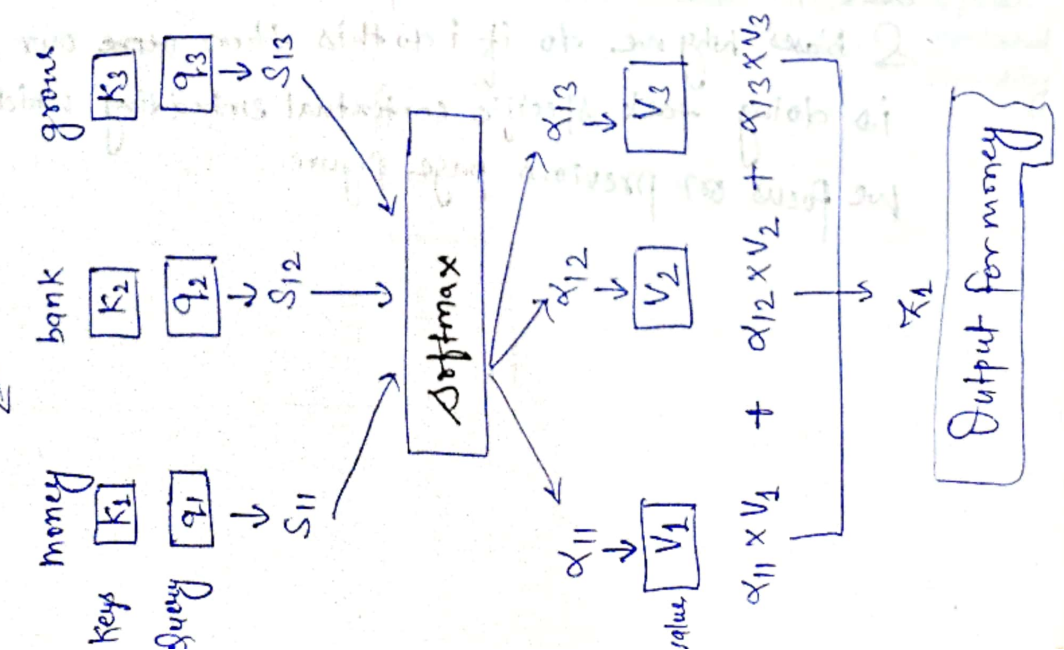
$$e_{\text{money}} = e_{\text{bank}}$$

$$e_{\text{money}} = e_{\text{grows}}$$

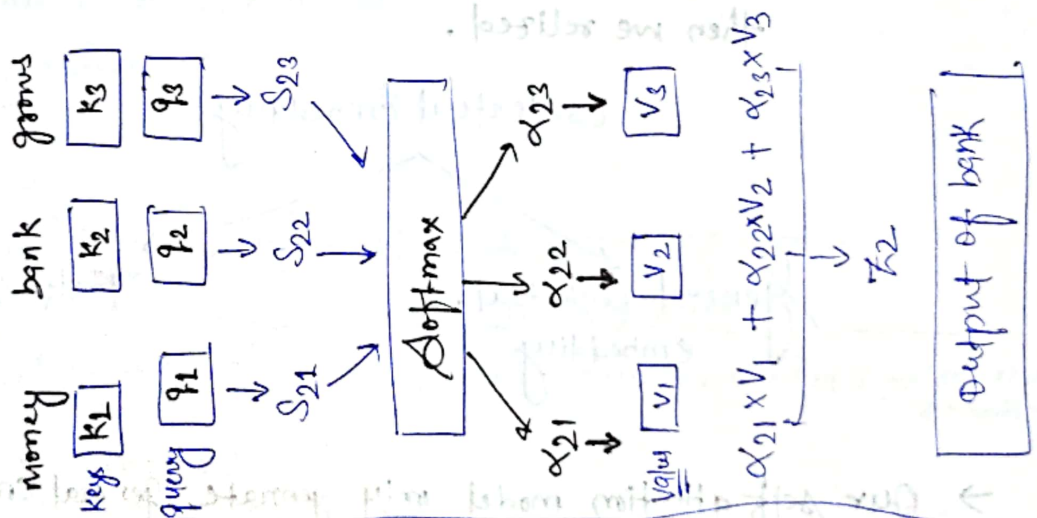
$$e_{\text{grows}} = e_{\text{money}}$$

Self Attention in Transformer :-

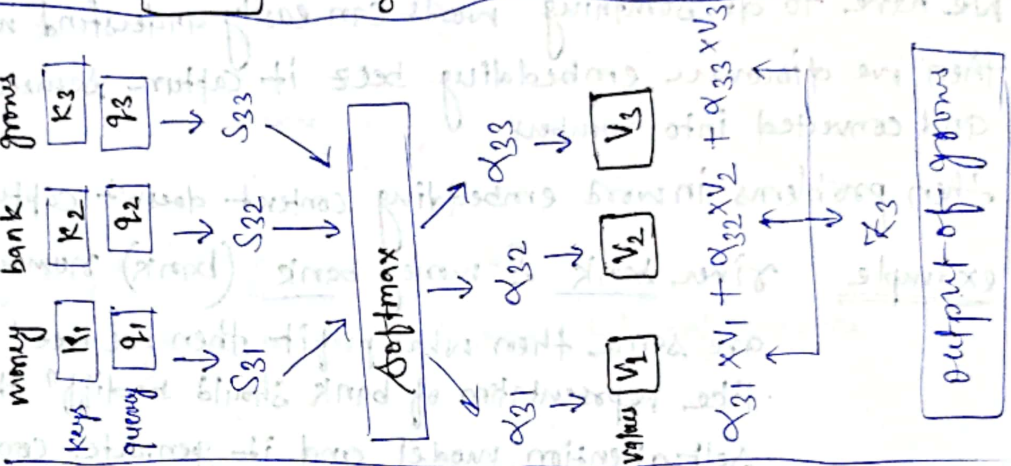
money x_1
Input embedding



bank x_2
Input embedding



groves x_3
Input embedding



V_i = Value vector derived from x_i

$S_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$

Scaled dot product

α_{ij} $\rightarrow$ Attention weight after softmax

X_i = Context aware representation (output)

$X_i = \sum_{j=1}^n \alpha_{ij} \cdot V_j$

$x_i \Rightarrow$ Input embedding

$q_i \Rightarrow$ Query vector derived from x_i

$K_i \Rightarrow$ Key vector derived from x_i

Points to consider :-

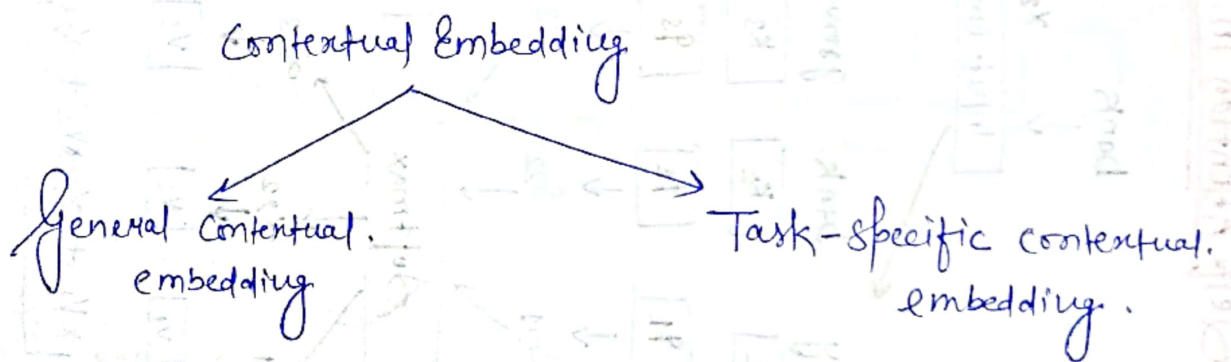
- This operation is a parallel operation
- There are no parameters involved.

Summary :-

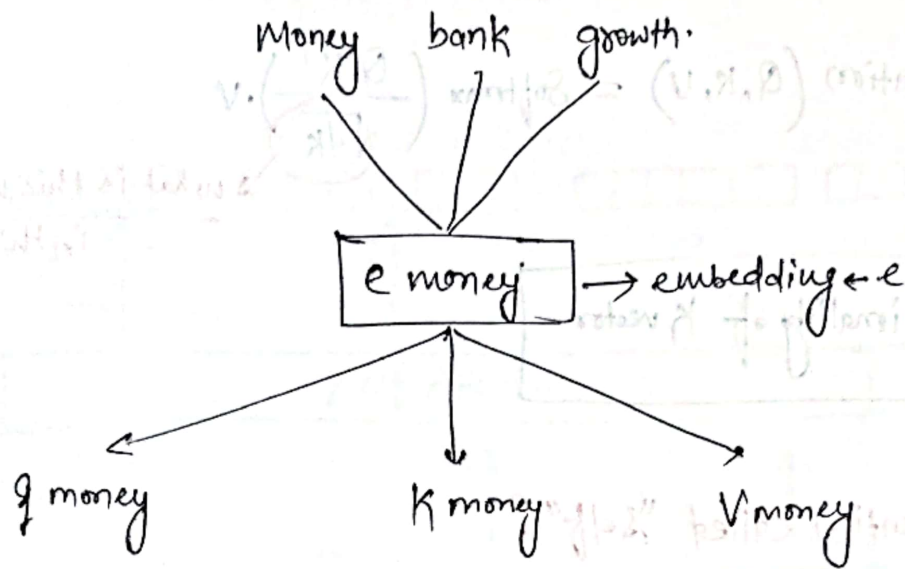
We have to do something words can easily understand machines.
then we discover embedding becz it captures semantic meaning of words
and converted into numbers.

then problems in word embedding context doesn't capture easily

example river bank & money bank (bank) numerical representation
are same, then what profit then we need contextual embedding
like representation of bank should be diffⁿ then we developed
self attention model and it generates contextual embedding
then we realized.



- Our self attention model only generates general contextual embedding becz there are no learnable parameters then we have to add some weights & biases why we do if I do this then our self attention model is doing task specific contextual embedding which we need then we focus on previous page figure.



where $k = \text{key}$, $q = \text{query}$, $v = \text{value}$

then Determine what information to look at and what info to take

#> Attention weights

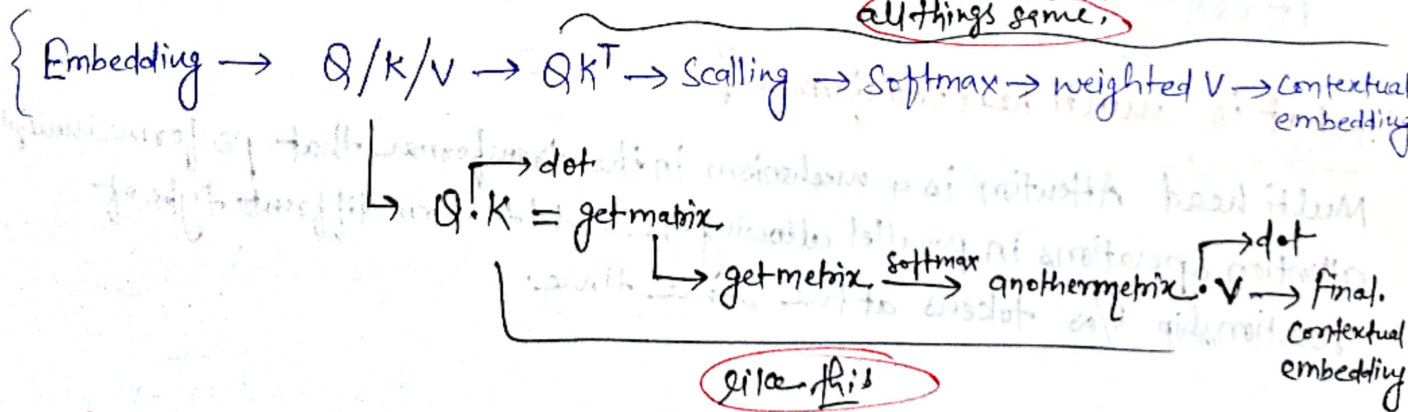
Determine how much each token matters

#> weighted combination of value

#> Contextual representation

"bank" now contains information influenced by its surrounding words.

all things same.



#> Formula to calculate Attention :-

$$\text{Attention}(Q, K, V) = \text{Softmax}(Q \cdot K^T) V$$

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

what is this we know?
in this

d_k = dimensionality of K vector

Q. Why is self Attention called "self"?

It is called self-attention because a sequence attends to itself using its own tokens to generate Q, K & V.

Self Attention → attention within the same sequence.

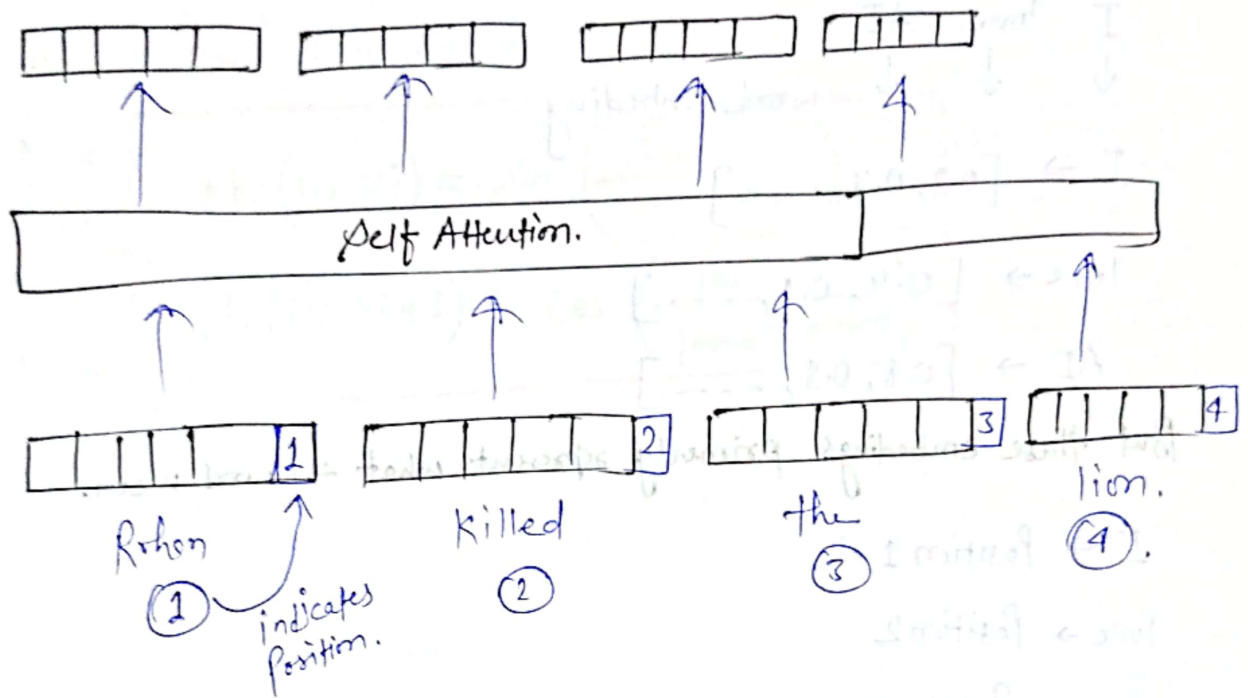
Cross Attention → One sequence attends to another sequence.

⇒ Problem in self attention is it figures out single prospective in 1 given sentence/document if multiple prospective happen it can't figure out.

#). What is multi head transformer?

Multi head Attention is a mechanism in the transformer that performs multiple attention operations in parallel allowing the model to learn different type of relationship b/w tokens at the same time.

#> Positional Encoding



Problems

1. Unbounded $\rightarrow$ upper limit (no. is huge)

$\rightarrow$ NN works on backprop and it takes huge no. $[-1 \text{ to } 1]$ it use.

Summary :-

Positional encoding is a way of giving a Transformer information about the positions/orders of tokens in a sequence.

The key problem is :-

Self attention doesn't inherently know the order of words.

like :-

Dog bites man

man bites dog

contain the same words but their meanings are completely different. A transformer needs to know which words comes first second third etc.

That's why positional encoding come in.

2. Why does a Transformer need it?

Consider

I lone AI
↓ ↓ ↓
word embedding

I → [0.2, 0.7, ...]

lone → [0.4, 0.1, ...]

AI → [0.8, 0.3, ...]

But these embeddings primarily represent what the word means,

I → Position 1

lone → Position 2

AI → Position 3

So combine them:-

Input representation = Token Embedding + Positional Encoding

This combined representation is then passed into the Transformer

Simple Intuition :-

You know Students in classroom but you don't know positions where is he sitting

Token embedding → what is this word?

Positional encoding → where is this word?

Embedding = meaning

Positional encoding = Position/order.

Q.3. How does the original Transformer do it?

⇒ The original Transformer paper used sinusoidal positional encoding.

It used sine & cosine functions:-

$$PE(Pos, 2i) = \sin\left(\frac{Pos}{10000^{2i/d_{model}}}\right)$$

$$PE(Pos, 2i+1) = \cos\left(\frac{Pos}{10000^{2i/d_{model}}}\right)$$

where:-

- Pos = position of the token
- i = dimension index
- d_{model} = embedding dimension

Q. Why sine & cosine?

⇒ The sinusoidal functions provide a continuous, structured representation of position the model to potentially learn relationship b/w positions and can generalize to sequence length beyond those seen during training.

Normalization

⇒ Normalization in deep learning refers to the process of ~~transformer~~ transforming data or model output to have specific statistical properties typically a mean of zero and variance of one.

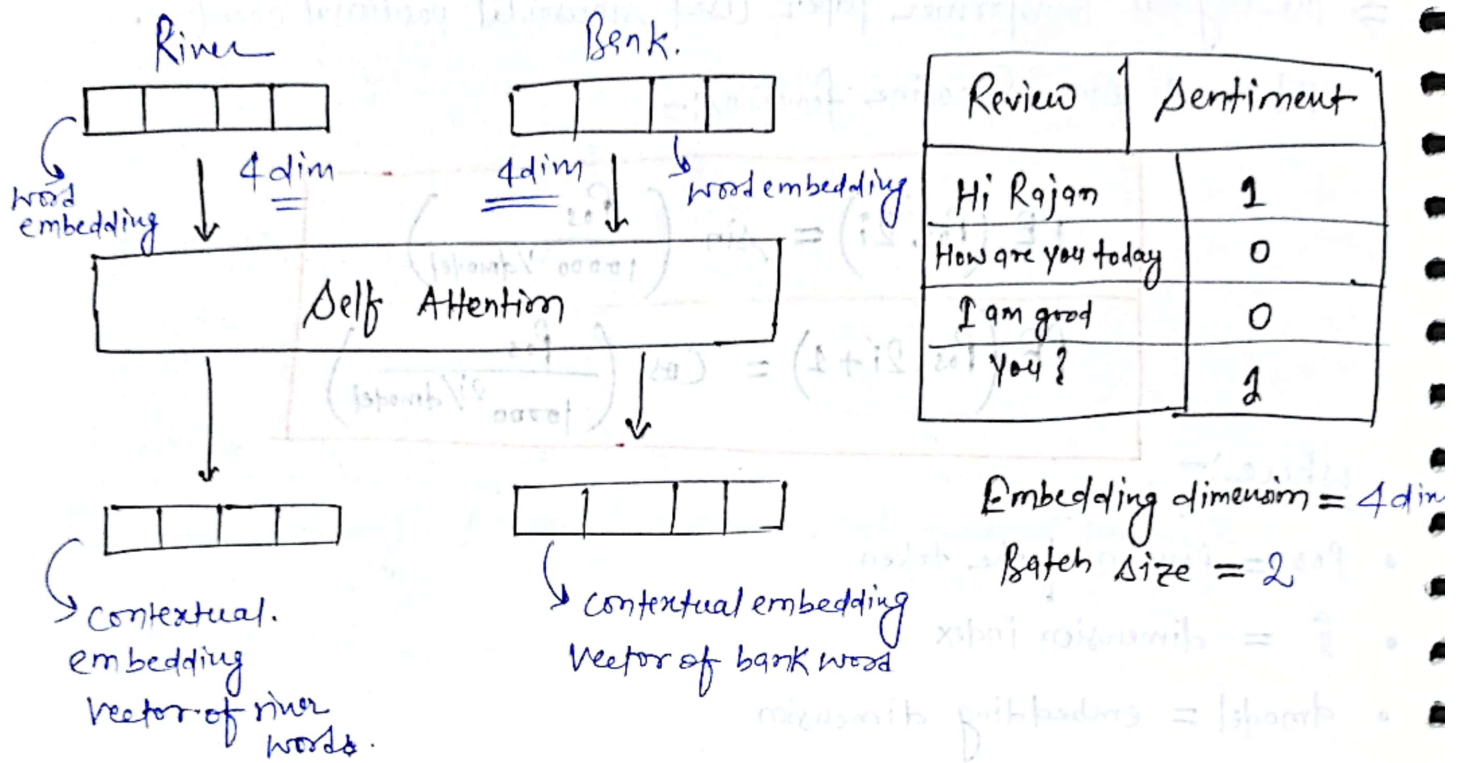
Benefits of Normalization in DL

1. Improve Training stability
2. faster convergence
3. Mitigating Internal covariate shift
4. Regularization Effect

Q. Why don't we use batch norm in Transformer

⇒ It doesn't work on sequential data.

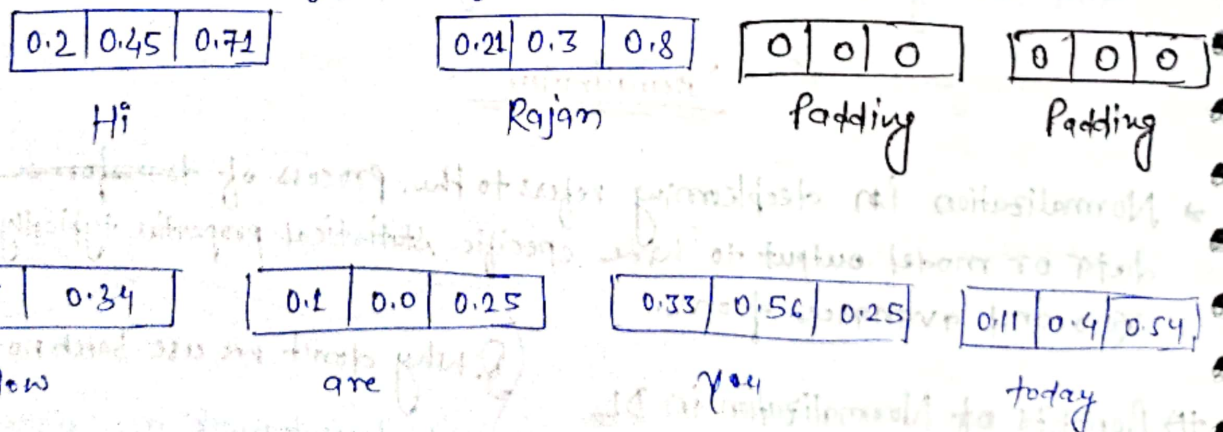
#>. Using batch Norm on Self Attention 2-



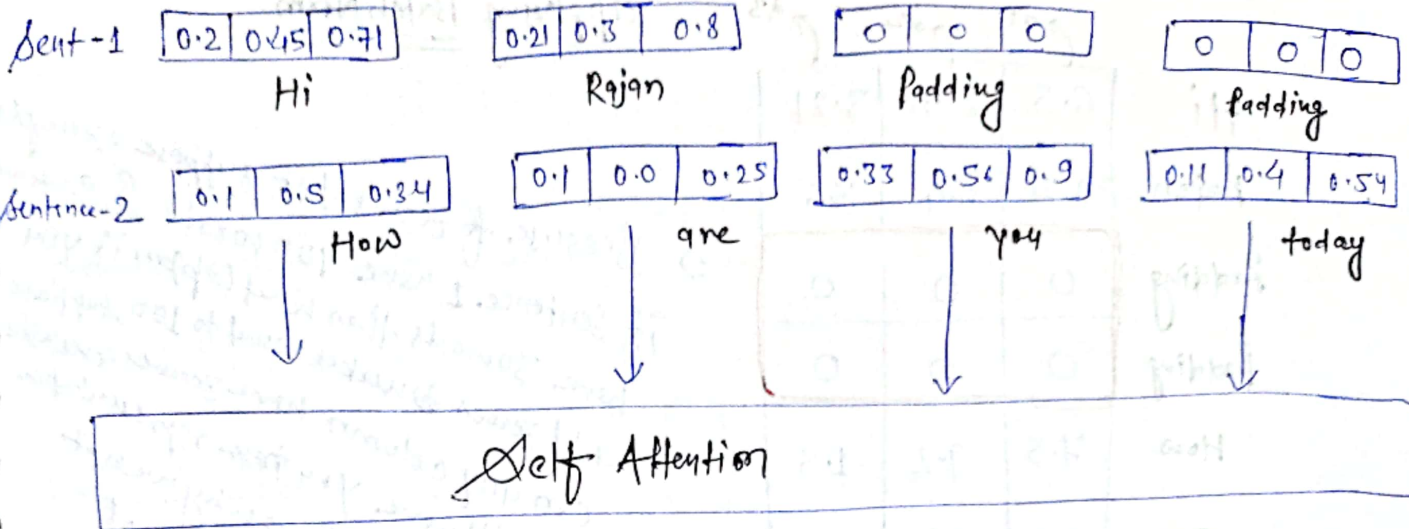
#>. You can send more than one sentence in self Attention using batches :-

#>. Embedding vectors :-

⇒ You can't focus on 0.2... this type of no. in box it is generated by model but now it is random no.



In self Attention if you sent this, this is not possible becz self attention accept same words (means no. of words in sentence) then we add padding



⇒ After that we get matrices calculating K, Q, V :-

⇒ now we generate it into matrix

Hi	0.2	0.45	0.71
Rajan	0.21	0.3	0.8
Padding	0	0	0
padding	0	0	0

(4x3)

→ actually it's single tensor (2, 4, 3)

How	0.1	0.5	0.34
are	0.1	0.0	0.25
you	0.33	0.56	0.9
today	0.11	0.4	0.54

(4x3)

Self attention.

Hi	6.5	2.41	3.21
Rajan	2.21	0.4	3.6
Padding	0	0	0
padding	0	0	0

How	7.5	9.2	1.5
are	2.2	1.1	6.7
you	2.9	6	9
today	9.9	2.3	6.5

Contextual embedding matrix

then we do normalization

Stackup both matrices.

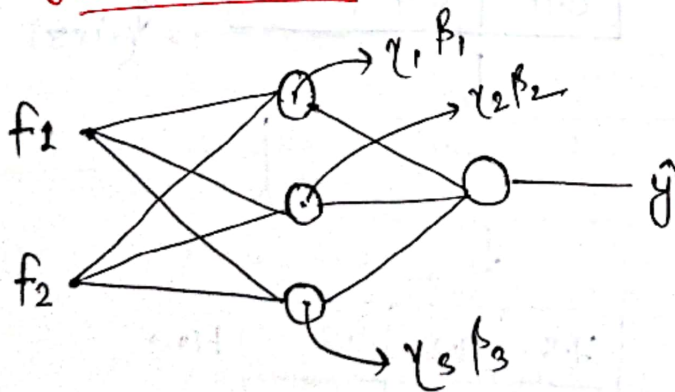
applying Batch Norm

	d_1	d_2	d_3
Hi	6.5	2.41	3.21
Byon	2.21	0.4	3.6
Padding	0	0	0
Padding	0	0	0
How	7.5	9.2	1.5
are	2.2	1.1	6.7
You	2.9	6	9
today.	9.9	2.3	6.5

⇒ Because of zeroes / let suppose example
if sentence 1 have 100 words & sentence 2
have 30 words then what happen if you
add zeroes & makes equal to 100 suppose
can any column have zeroes & make
it will give you true representation
no. if cant. this is why we not
use Batch Norm in case of
self attention or transformer.
this problem is placed in
sequential data.

μ_1, σ_1
 γ_1, β_1
 μ_2, σ_2
 γ_2, β_2
 μ_3, σ_3
 γ_3, β_3

Layer Normalization :-



f_1	f_2	z_1	z_2	z_3
2	3	7	5	4
1	1	2	3	4
5	4	1	2	3
6	1	7	5	6
7	1	3	3	4

μ_1 mean
 σ_1 Standard deviation

$$\text{Normalized} = \frac{7 - \mu_1}{\sigma_1} = 0.3\gamma_1 + \beta_1$$

$$= \frac{5 - \mu_1}{\sigma_1} = 2.2\gamma_1 + \beta_1$$

$$= \left(\frac{5 - \mu_4}{\sigma_4} \right) \gamma_2 + \beta_2$$

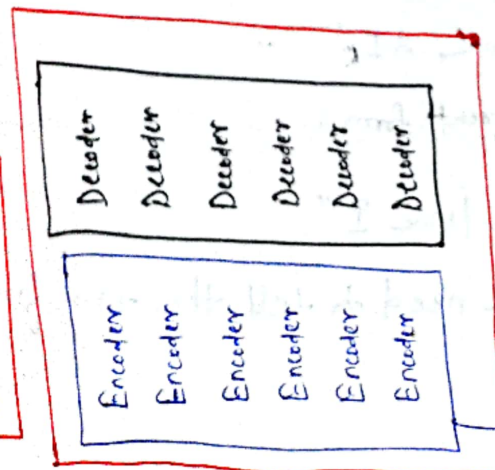
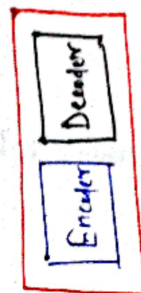
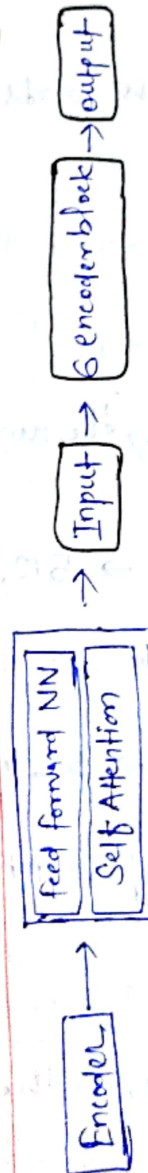
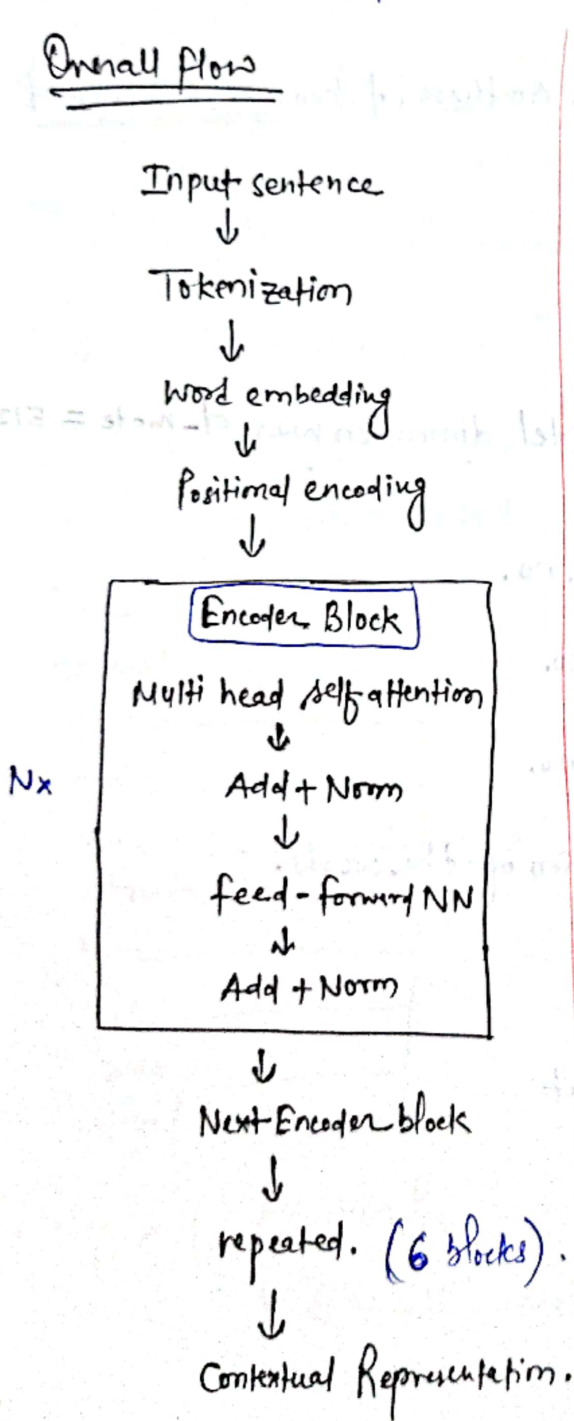
Transformer Architecture

Encoder

Think a transformer Encoder as a pipeline that takes a sentence and gradually converted each word into a context-aware representation.

For ex- "The bank approved the loan"

Overall flow



Every blocks are identical & parameters are different

2. Tokenization :-

First sentence is converted into tokens.

Eg:- I love AI

["I" "love" "AI"]

then, the model can't directly process words so these tokens are mapped to numerical representation.

2. Word Embedding :-

Each tokens becomes a vector

for Original Transformer architecture, the model dimension was $d_{\text{model}} = 512$

"I" $\rightarrow [0.12, -0.42, \dots] \rightarrow 512 \text{ no.}$

"love" $\rightarrow [0.72, 0.11, \dots] \rightarrow 512 \text{ no.}$

"AI" $\rightarrow [0.51, -0.33, \dots] \rightarrow 512 \text{ no.}$

Now Computer has numerical representation of the words.

3. Positional Encoding :-

Self-Attention doesn't inherently know that

"I love AI"

is different from,

"AI love I"

So we need to tell the transformer where each token occurs.

Word embedding + Positional Encoding = Position-aware embedding

for eg:-

I + Position 1
love + Position 2
AI + Position 3

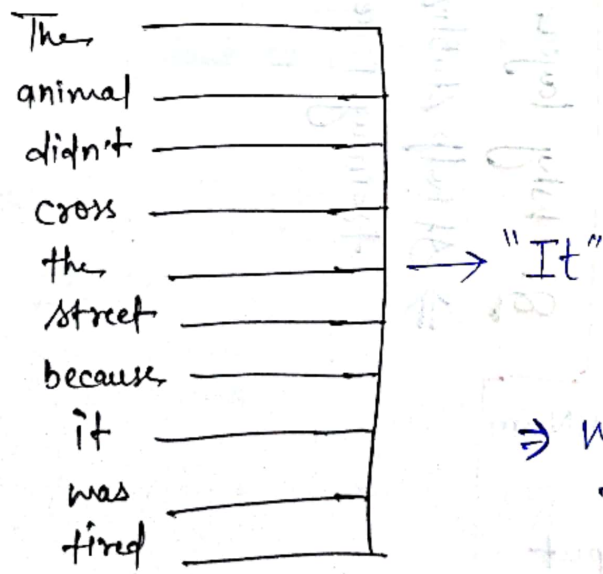


4. Multihead Attention :-

Eg Suppose:- The animal didn't cross the street because it was tired.

⇒ when the model process "it", it needs to understand what it refers to.

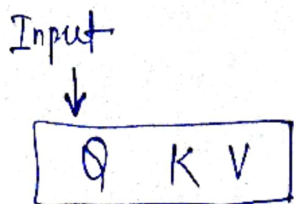
Self attention allows the model to look at the others words and determine which ones are important.



⇒ model learns relationship b/w these tokens

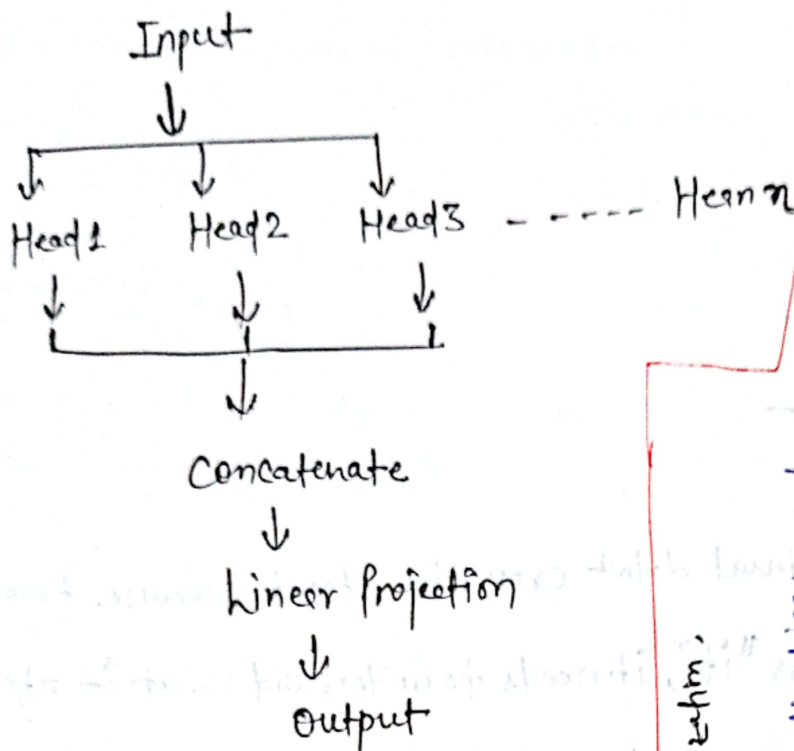
Q. Why self Attention?

Because Q, K & V all comes from the same input sequence



↓
Self Attention.

⇒ And we use multiple attention heads:-



5. Add & Norm :-

Input = X

Attention output = Z

then conceptually :-

Attention $(X) = Z \rightarrow \text{ADD} \rightarrow \text{Layer Norm}$
 $\downarrow$
 Output

Mathematically :-

$$\text{Output} = \text{LayerNorm}(X + Z)$$

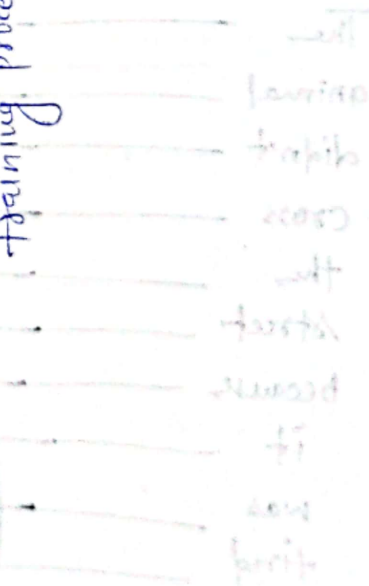
Q. Why the shortcut?

It helps information and gradients flow through deep networks more easily.

Q. Why layer Normalization?

⇒ It helps stabilize the activations and training process.

training process



6. feed-forward Network :-

512 dimension representation

↓ Expand

2048 dimension representation

↓ Non-linear Processing → compress

512 dimension representation.

7. Second Add & Norm :-

Previous representation →

Add ← FNN Output

↓

layer Norm

↓

Output

$$\text{Output} = \text{layer Norm}(\text{Input} + \text{FNN}(\text{Input}))$$

Summary :- Transformer Encoder :-

Input

↓

Tokenization

↓

Embedding + Positional Encoding

↓

Encoder Block x 6

→ multihead self Attention

→ Residual connection

→ layer Normalization

→ feed forward Network

↳ 512 - 2048 → 512

→ Residual connections.

→ layer Normalization.

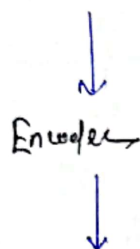
→ Contextual Representation.

#> Encoder Vs Decoder

Encoder :-

The encoder receives the complete input sequence.

Eg:- "I love India"



Contextual representation

It can use normal self-attention because it is allowed to see the whole input.

Decoder :-

The decoder generates

"I"



"I love"

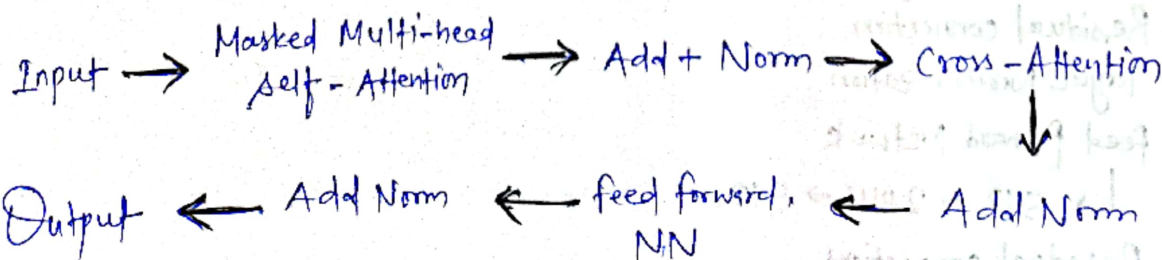


"I love India"

It cannot look at future output tokens. That's why it uses "Masked Self-Attention".

2. Transformer Decoder Architecture :-

★ decoder block has three major components.



This is the most important diagram to remember, (in previous page).

3. first:- Masked Self-Attention :-

Let suppose the decoder has :-

"I love machine learning"

When Predicting machine it can see.

I ✓
love ✓
machine ✓
learning x

So, attention looks approximately like:-

⇒

	I	love	machine	learning
I	✓	x	x	x
love	✓	✓	x	x
machine	✓	✓	✓	x
learning	✓	✓	✓	✓

The future tokens are masked, but why?

→ Because during generation :-

I
↓
I love
↓
I love machine
↓
I love machine learning

The model doesn't know the future

So masked self-attention makes training behave like generation.

4. Cross - Attention :-

Here the decoder communicates with encoder.

Encoder



Encoder Output

The decoder takes the encoder output and ask:-

→ Which parts of the input should I focus on while generating my next word?

Input

I love India.

Encoder → Encoder representation → Cross Attention → Decoder Representation.

The important Q, K, V difference

In self attention

Q ← Decoder

K ← Decoder

V ← Decoder

In Cross attention

Q ← Decoder

K ← Encoder

V ← Encoder

★ Query comes from decoder while key and value comes from encoder.

⇒ That's why cross-attention allow the decoder to retrieve relevant important from the input sequence.

5. feed forward Network :- (FFN)

After cross attention.



Add Norm



feed forward Network

The FFN processes each position independently

In the original Transformer architecture

512 $\rightarrow$ 2048 $\rightarrow$ 512

Conceptually :-

Input $\rightarrow$ Linear $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ Output

6. Complete Decoder Block :-

Decoder Input



Masked Multi-head
Self-attention.



Add Norm



Cross Attention

← Encoder Output



Add Norm



feed forward NN



Add Norm



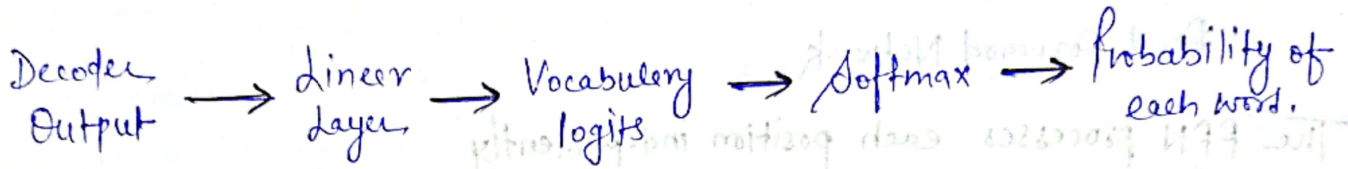
Decoder Output

The original Transformer used a stack of 6 decoder layers with each layer containing these sublayers.

7. finally linear layer + Softmax :-

⇒ The decoder's output isn't directly a word.

⇒ It is converted into probabilities over the vocabulary



Example

I love _____

India	0.62	} ⇒ The model select/predicts the next token based on these probabilities.
Python	0.08	
Machine	0.15	
Football	0.03	

8. How Generation Actually Happens :-

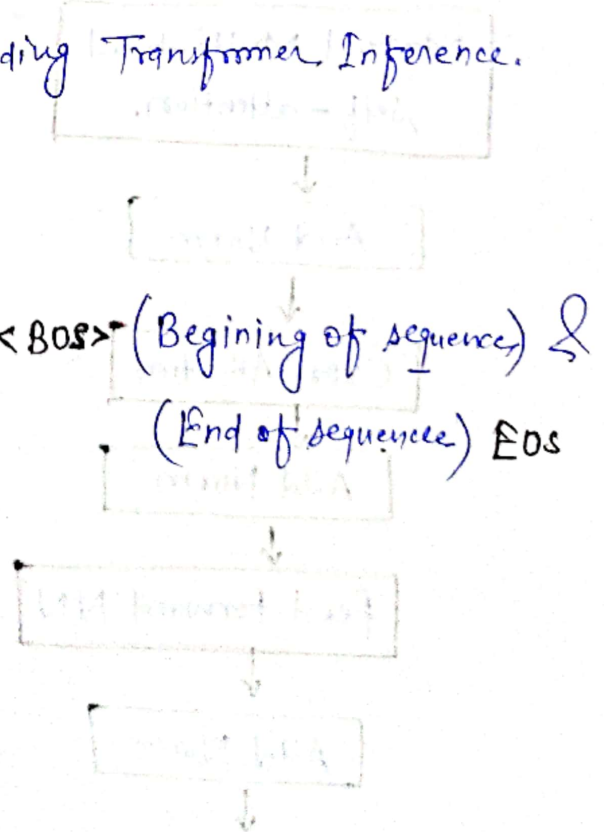
This is extremely important for understanding Transformer Inference.

Suppose we want :-

"I love India"

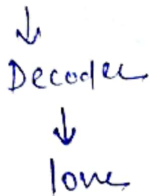
The decoder starts with a special tokens $\langle \text{BOS} \rangle$ (Beginning of sequence) & $\langle \text{EOS} \rangle$ (End of sequence)

Step 1 $\langle \text{BOS} \rangle$
↓
Decoder
↓
"I"



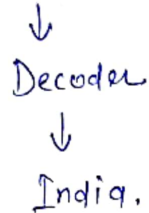
Step 2

<BOS> I



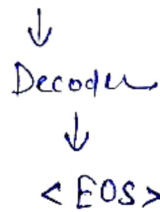
Step 3

<BOS> I love



Step 4

<BOS> I love India.



⇒ Each prediction becomes part of the input of the next prediction.

⇒ Transformer Decoder :-

The Transformer decoder generates the target sequence autoregressively. Each decoder layer contains masked multi-head self attention, cross attention and position-wise feed forward neural network with residual connections and layer normalization. Masked self attention allows the decoder to attend only to previous and current tokens, preventing it from seeing future tokens. In cross attention, queries come from the decoder while keys and value come from encoder output, allowing the decoder to focus on relevant information from the input sequence. Finally, the output is passed through a linear layer and softmax to predict the next tokens.

Prone